

PA2 Report

Fixed-outline Floorplanning

Student: 楊澤全

WEB ID: pd26s092

1. Introduction

This project solves the fixed-outline floorplanning problem. Given a set of **movable hard blocks, fixed terminals, and a netlist**, the goal is to place all blocks inside a prescribed outline while **minimizing a weighted combination of area and wirelength**.

The final objective reported to the evaluator is:

$$\text{ReportCost} = \alpha * \text{Area} + (1 - \alpha) * \text{Wirelength}$$

- Area is the area of the final packed bounding box.
- Wirelength is the sum of HPWL over all nets.

In this implementation, fixed terminals use their exact coordinates, while each movable block contributes the center of its placed bounding box to HPWL evaluation.

A legal solution must satisfy the following geometric conditions:

- All blocks are non-overlapping
- All blocks lie within the fixed outline.

Throughout the search, every placement is reconstructed from the current B*-tree by contour-based packing.

2. Data Structure

2.1 Terminal, Block, and Net

The lowest-level data structures are defined in module.h.

Terminal stores a named geometry endpoint. For **fixed terminals**, the stored coordinates represent a **single point**. For movable blocks, the same coordinate fields are reused to store the placed bounding box.

Block **inherits** from Terminal and adds:

- ◆ Original width and height
- ◆ Rotation flag

This design keeps placement geometry and netlist connectivity consistent, because both fixed terminals and placed blocks can be handled through the same Terminal interface.

Net stores **a vector of Terminal pointers**. This allows the same net to connect arbitrary combinations of fixed terminals and movable blocks. **HPWL is computed directly from the current geometry of all connected endpoints.**

2.2 B*-Tree Representation

The floorplan topology is represented by an index-based B*-tree:
struct BStarNode

```
{  
    int blockIdx;  
    int parent;  
    int left;  
    int right;  
};
```

The implementation follows the standard B*-tree geometric interpretation:

- ◆ Left child => Placed to the right of its parent
- ◆ Right child => Stacked above the parent region according to the current contour.

Each tree node stores only **index** rather than **pointer**. This simplifies copying, rollback, subtree edits, and swap operations during SA.

2.3 Contour Structure

Packing is performed with a contour structure implemented as a linked list over an array:

```
struct ContourSegment  
{  
    int front;  
    int back;  
};
```

Each active contour segment corresponds to one placed block. During packing, the solver scans the contour to find the **highest supporting y-coordinate** over the horizontal span of a new block, places the block there, and then **updates the contour** links accordingly.

This contour representation avoids explicit geometric intersection checks among all blocks and reconstructs a non-overlapping placement directly from the B*-tree.

2.4 Floorplanner

The Floorplanner class owns all benchmark data and all mutable search state. Its major containers include:

```
vector<Block*> _blkList  
vector<Terminal*> _tmlList  
vector<Net*> _netList  
vector<BStarNode> _nodes  
vector<ContourSegment> _contour
```

Additional **hash maps connect names to blocks, terminals, and tree indices**. This combination gives the solver both fast access by object identity and efficient topology updates.

2.5 SolutionState

SA frequently needs to compare, preserve, and restore previous placements. For this reason, the solver defines a **dedicated snapshot structure**:

```
struct SolutionState
{
    vector<BStarNode> nodes;
    vector<bool> rotations;
    vector<array<size_t, 4>> blockPositions;
    int rootIdx;
    size_t packedWidth;
    size_t packedHeight;
    size_t area;
    size_t overflowWidth;
    size_t overflowHeight;
    double wireLength;
    double reportCost;
    double searchCost;
    bool feasible;
    bool valid;
};
```

Each snapshot stores:

- ◆ Complete B*-tree topology
- ◆ Orientation of every block
- ◆ Placed coordinates of every block
- ◆ Objective values.

This design supports **efficient rollback and best-state tracking**. Snapshot capture and restore are $O(1)$.

2.6 GaussianStats

To compare feasible solutions more robustly, the solver also maintains a simple Gaussian model:

- ◆ Mean and standard deviation of feasible areas
- ◆ Mean and standard deviation of feasible wirelengths
- ◆ Validity flag

This model is not used for move acceptance. Instead, it is **used to rank feasible solutions and to detect whether successive outer-round solution pools are still improving.**

3. Flow Overview

3.1 Input Parsing

The program begins in main.cpp. It reads:

- alpha
- block file
- net file
- output file

During Floorplanner constructor net parsing, the solver also accumulates the net **degree of each block**. This information is later used to **bias perturbation selection toward more important blocks.**

3.2 Seed Selection

Before floorplanning starts, the solver chooses a **deterministic random seed.**

Each seed runs a complete copied planner, and the **best result is selected** by the following preference order:

- feasible over infeasible
- smaller overflow if infeasible
- lower reported cost if feasible
- then smaller area and wirelength

3.3 Initial Topology Construction

After parsing, the planner initializes a B*-tree.

The main initialization procedure constructs a bottom-left-anchored tree, **which larger blocks are placed earlier.**

This arrangement keeps large macros near the root and produces **more stable initial packings** than a purely random tree.

3.4 Initial Multi-Start Solutions

One seeded annealing run does not start from a single placement. Instead, it builds multiple initial solutions.

The current implementation uses:

- 2 fixed local seeds
- 3 rotation modes per seed

Therefore 6 initial states in total.

The process is:

1. Sort blocks by **area**, with additional bias from net degree and outline shape
2. Shuffle subsets of the remaining blocks under the **local seed**
3. Rebuild the B*-tree and apply one of **three rotation modes**:
 - normal
 - horizontal bias
 - vertical bias
4. Pack and evaluate the resulting placement.

The purpose of this design is to expose the solver to **different structural and orientation patterns** before SA, thereby reducing the chance of being trapped by one poor starting point.

3.5 Packing and Evaluation

Given a B*-tree and block orientations, packing converts topology into concrete block coordinates.

The packing procedure starts at the root:

- the root is placed at the origin
- each left child starts from the right edge of its parent
- each right child starts from the x-position of its parent and is lifted by the contour

After all nodes are placed, the solver scans all blocks to obtain:

- **packed width**
- **packed height**
- **packed area**
- **overflow width and height**
- **HPWL**

Two objective values are then maintained:

$$\text{ReportCost} = \alpha * \text{Area} + (1 - \alpha) * \text{Wirelength}$$

$$\begin{aligned} \text{SearchCost} = & \alpha * \text{normalizedArea} + (1 - \alpha) * \text{normalizedWire} \\ & + \text{overflowPenalty} \end{aligned}$$

4. Multi-Stage Simulated Annealing

The complete optimization flow is summarized below.

Step	Main Goal	Key Behavior	Output
Broad Exploration	diversify the search	➤ loose overflow tolerance ➤ more exploratory move ➤ high acceptance rate	Stage-1 anchors from each initial state

Legalization	reduce overflow	➤ increasing overflow penalty ➤ simultaneously explore low overflow states	Lower-overflow
Refinement	improve feasible cost	➤ Lower temperature ➤ more targeted rotate/swap	High-quality feasible candidates
Diversification	escape local minima	➤ kick moves and selective ➤ extra search on Refinement states	Refreshed strong chains
Pool Update	keep only good states	➤ rank by feasibility, cost, overflow ➤ recompute normalization and Gaussian statistics	Select Top pool for the next outer round

The solver first runs the Broad Exploration stage from all **six initial states**, collects the resulting anchors, and prunes them into a bounded solution pool. It then performs 4 or 5 outer rounds. In each round, one annealing chain is restarted from every surviving pool state, followed by **pool re-ranking, pruning, re-normalization, and Gaussian-statistics refitting**. Early stopping is triggered when the new pool no longer shows meaningful improvement.

Three common rules apply throughout this flow.

➤ Search cost

The search objective is

$$\text{SearchCost} = \alpha * \text{normalizedArea} + (1 - \alpha) * \text{normalizedWire} + \text{overflowPenalty}$$

where the overflow penalty becomes increasingly **strict in later stages**.

➤ Rollback policy

Each stage keeps track of three representative states: the best search-cost state (bestSearch), the lowest-overflow state (bestOverflow), and the best feasible state (bestFeasible). When the stage ends, the solver does not simply keep the last visited state. Instead, it rolls back to the most desirable anchor available, **preferring a feasible solution first, then the lowest-overflow solution, and finally the best search-cost solution**.

➤ Gaussian feasible-state ranking

Feasible states are compared using a **Gaussian-normalized score based on the z-scores of area and wirelength**. This Gaussian model is used only for **feasible-state ranking and outer-round convergence checking**, not used for move acceptance.

5. Main Design Improvements

5.1 Multi-Start Initialization

Using 2 local seeds and 3 rotation modes creates 6 structurally **different starting placements** per seeded SA run. This is much more robust than relying on one initial B*-tree.

5.2 Anchored Initial Tree Construction

Instead of constructing a purely random tree, the solver deliberately places **large blocks near the lower-left** base and spreads later blocks outward. This reduces early fragmentation and yields more stable packings.

5.3 Stage-Adaptive Search Cost

The search objective is legality-aware. In **early stages** it tolerates **moderate overflow to encourage exploration**, and in **later stages** it **tightens the overflow threshold** and increases penalty strength to pull the search into the feasible region.

5.4 Heuristic Perturbation Ranking

Candidate ranking makes the search more **informed without becoming fully greedy**. The solver still retains **stochastic** behavior, but more **promising edit** receive a higher probability of being selected.

5.5 Gaussian-Based Feasible Comparison

The Gaussian model helps compare feasible solutions on a relative scale within the current pool. This is particularly useful when area and wirelength have different numeric ranges.

6. Complexity Analysis

Let:

- n be the number of blocks
- p be the total number of net pins across all nets

6.1 Packing and Evaluation

One HPWL evaluation costs $O(p)$, since every net endpoint is visited once.

One packing pass is $O(n^2)$ in the worst case, because each block may scan a long prefix of the contour while being placed. (**subtree contour $O(n)$**)

Therefore, one full state evaluation costs **$O(n^2 + p)$** .

6.2 Annealing Cost per Seeded Run

For each temperature level, the solver performs $O(n)$ move attempts because the

move count is proportional to the number of blocks.

One stage of one chain costs: $O(n(n^2 + p)) = O(n^3 + np)$.

The number of **initial states**, **stages**, **outer rounds** are all fixed small constants in this implementation. Therefore, one full seeded annealing run remains in $O(n^3 + np)$, with a relatively large but bounded constant factor

7. Result

Case	Area	Wirelength	Runtime	Quality Score	Runtime Score	Total
ami33	1,252,636	85,492	5.65	7.66	6.26	7.38
ami49	37,611,028	1,063,496	13.93	9.37	6.57	8.81
apte	47,313,280	748,244	0.59	8.37	6.97	8.09
hp	9,574,992	258,006	0.69	8.31	6.99	8.05
xerox	20,088,040	510,819	0.91	8.53	6.83	8.19

8. 討論與心得

這題本身是我參考課程提供的 B*tree 的 source code，並加上 SA 的一些擾動來做延伸。但單純只用這兩種方法，效果很有限，因為很大取決於運氣成分。(參數或初始解)

因為先前自己實作時，結果真的很差，隨機性太高，很難得到好的結果。所以我有搭配半隨機方法(最好的選擇有更大概率被選擇到)、Outer 與 Inner Seed、Multi-start 等方法。

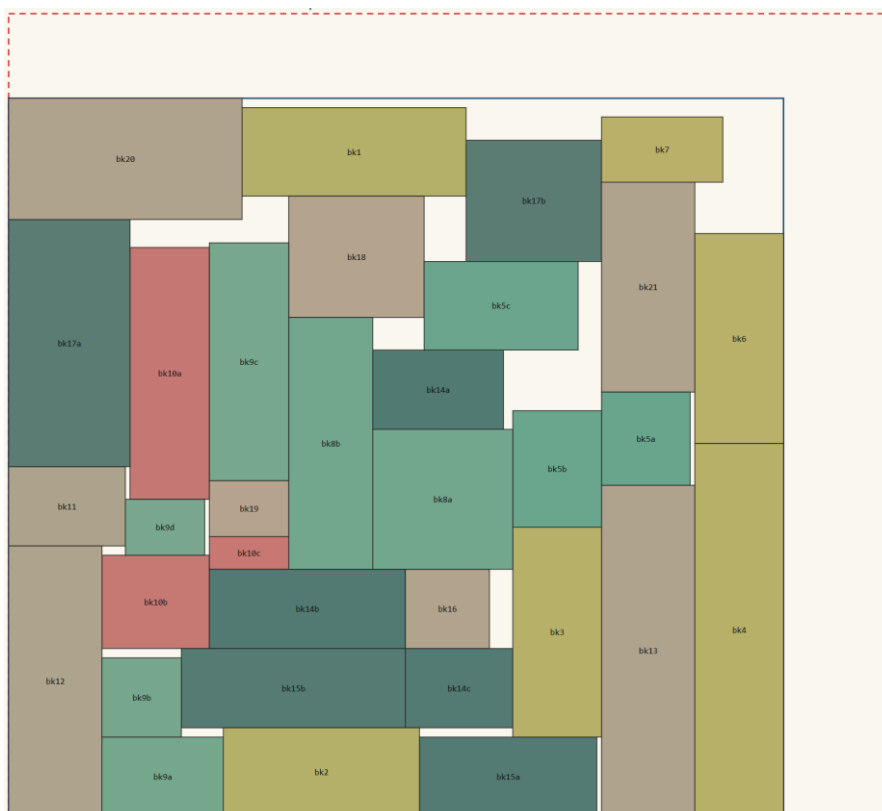
所以我後來加入了許多 heuristic 的條件。例如 SA 移動時需要根據關係來選擇：

1. 當選擇要移動過去的 block 時，要根據條件選到更理想的 block，像是優先考慮它是否在邊界位置和本身大小等。
2. 如果要進行交換，會傾向選擇兩個大小相似的 block，這樣交換後比較不會產生問題。
3. 在後期微調的時候，如果將旋轉搭配交換，或是選擇搭配移動，可能會有更好的解和突破。

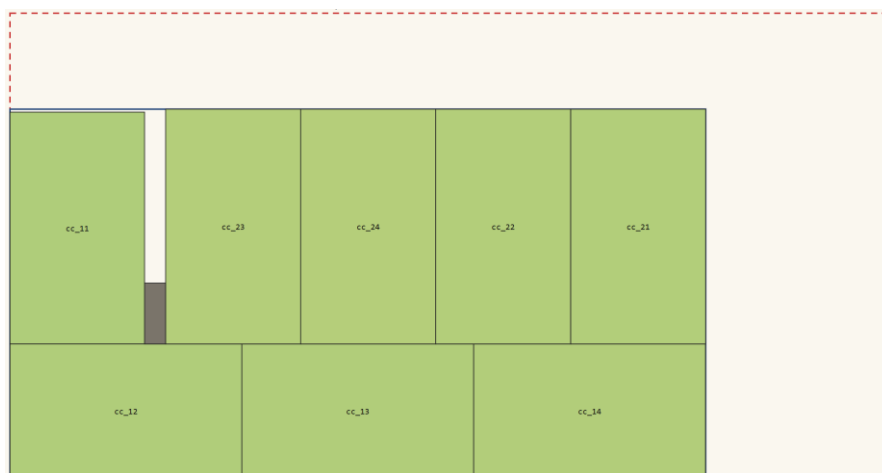
我有嘗試推測要用什麼當作篩選條件，但很多數值我沒甚麼想法，蠻多是請 AI 幫我 tune 的。但也從一開始有個 Case 完全塞不下、怎麼動都放不進去，到後來優化出還算不錯的結果，確實花了很久的時間。

我覺得 SA 的部分還是很看運氣，因為參數大多是 heuristic 的，目前很難有嚴謹的數學證明。後續可能要再慢慢摸索如何更嚴謹證明，畢竟現在很多參數是 AI 給我的經驗數值，我也只能透過調整參數跑跑看，以目前的結果來說還算能接受。

Ami33 看起來還是蠻多 dead space，可以再改進。



Apte 目前跟最小面積相同，但線長可以再縮減。



Ami49 一開始完全放不下，但現在意外發現大 block 都擠在右上方。

